Appendix: R Code — How Did Rate Hikes Affect Housing Supply?

STAT-4840 | Time Series Analysis

Katrina Churchill

April 2026

Contents

1 Load Packages

```
library(TSA)
library(forecast)
library(tseries)
library(ggplot2)
library(dplyr)
library(lubridate)
library(readr)
library(zoo)
library(lmtest)
library(knitr)
```

---

2 Load and Clean Data

Data source: FRED (Federal Reserve Bank of St. Louis), Series HOUST1F. New Privately-Owned Housing Units Started: 1-Unit Structures. Monthly, Seasonally Adjusted Annual Rate, in thousands. Original data from U.S. Census Bureau / HUD.

```
# Data source: FRED Series HOUST1F
# Original URL: https://fred.stlouisfed.org/graph/fredgraph.csv?id=HOUST1F
# To refresh data, run in console:
#   raw <- read_csv("https://fred.stlouisfed.org/graph/fredgraph.csv?id=HOUST1F")
#   write_csv(raw, "HOUST1F.csv")

raw <- read_csv("HOUST1F.csv", show_col_types = FALSE)
names(raw) <- c("date", "starts")

housing <- raw %>%
  mutate(date   = as.Date(date),
         starts = as.numeric(starts)) %>%
  filter(date >= as.Date("2015-01-01"),
         date <= as.Date("2025-03-01"),
         !is.na(starts))

cat("Observations:", nrow(housing), "\n")
```

```
## Observations: 123
```

```
cat("Range:", format(min(housing$date), "%b %Y"),
    "to", format(max(housing$date), "%b %Y"), "\n")
```

```
## Range: Jan 2015 to Mar 2025
```

```r
# Convert to monthly ts object
starts_ts <- ts(housing$starts, start = c(2015, 1), frequency = 12)
```

---

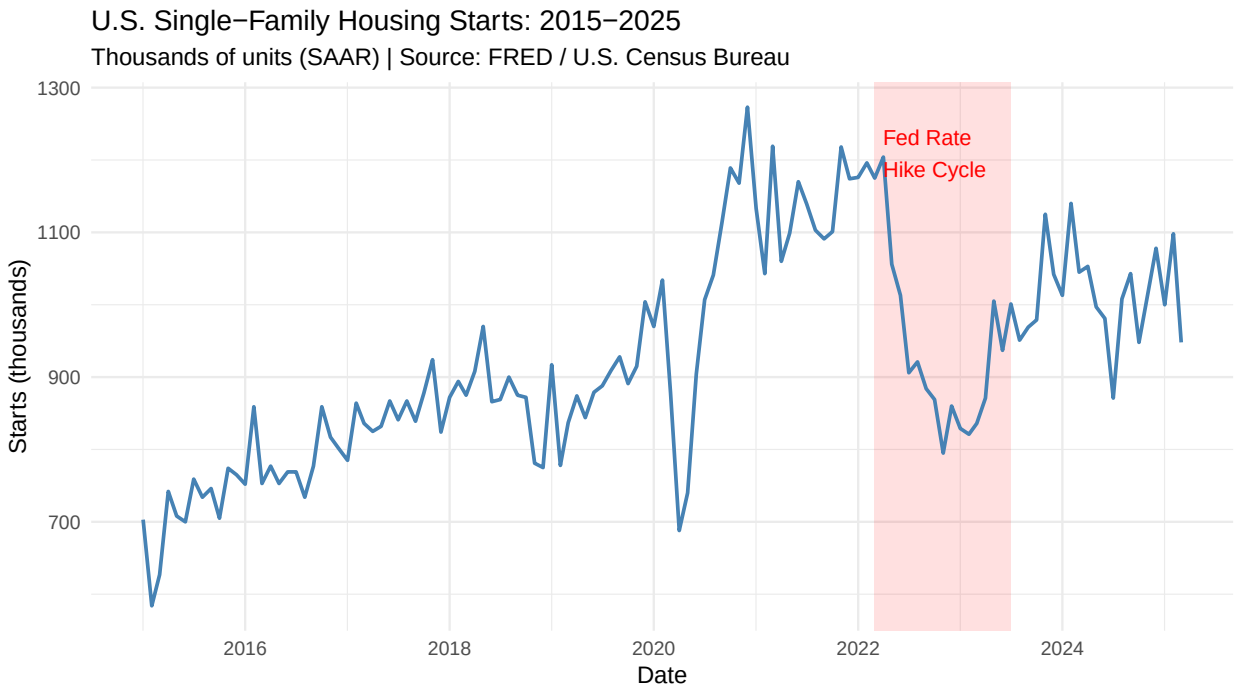
3 Section 1 — Preliminary Analysis

3.1 Time Series Plot

Red shading marks the Federal Reserve rate hike cycle (March 2022 - July 2023).

```r
hike_start <- as.Date("2022-03-01")
hike_end   <- as.Date("2023-07-01")

ggplot(housing, aes(x = date, y = starts)) +
  annotate("rect", xmin = hike_start, xmax = hike_end,
           ymin = -Inf, ymax = Inf, fill = "red", alpha = 0.12) +
  geom_line(color = "steelblue", linewidth = 0.8) +
  annotate("text", x = hike_start + 30,
           y = max(housing$starts) * 0.95,
           label = "Fed Rate\nHike Cycle",
           color = "red", size = 3.5, hjust = 0) +
  labs(title = "U.S. Single-Family Housing Starts: 2015-2025",
       subtitle = "Thousands of units (SAAR) | Source: FRED / U.S. Census Bureau",
       x = "Date", y = "Starts (thousands)") +
  theme_minimal()
```



3.2 Summary Statistics

```
summary(starts_ts)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   584.0   824.5   891.0   922.6  1024.0  1273.0
```

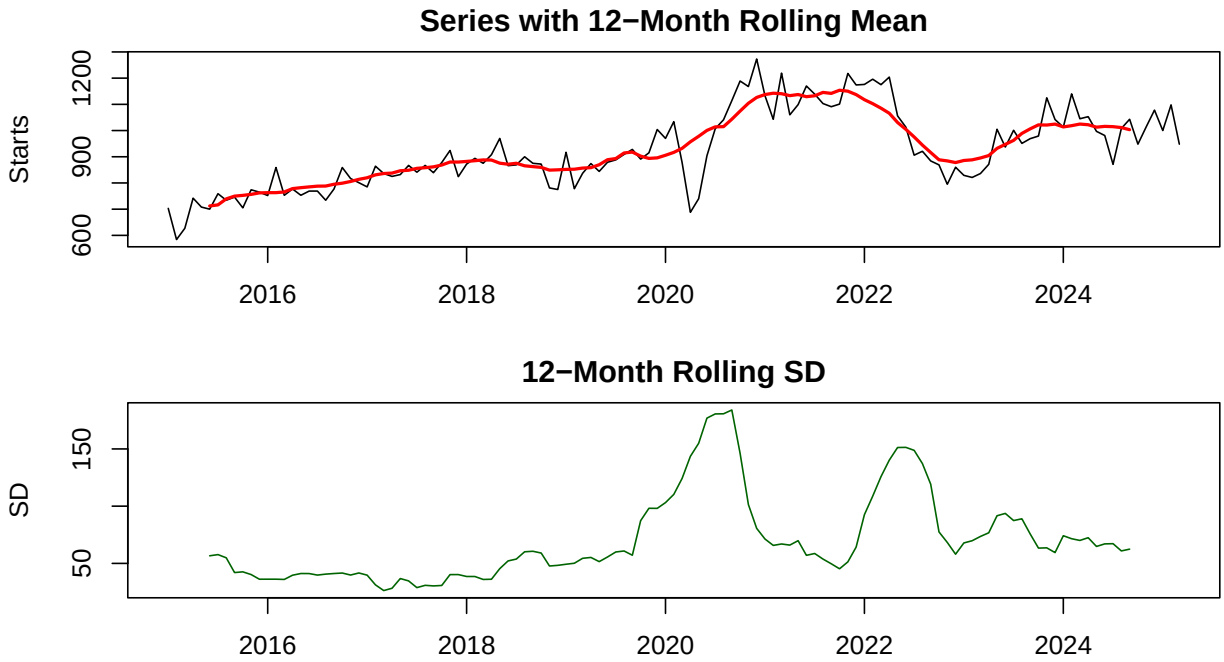
```
cat("Std Dev:", round(sd(starts_ts), 2), "\n")
```

```
## Std Dev: 145.79
```

3.3 Rolling Mean and SD

```
roll_mean <- rollmean(starts_ts, k = 12, fill = NA)
roll_sd   <- rollapply(starts_ts, width = 12, FUN = sd, fill = NA)

par(mfrow = c(2, 1), mar = c(3, 4, 2, 1))
plot(starts_ts, main = "Series with 12-Month Rolling Mean", ylab = "Starts")
lines(roll_mean, col = "red", lwd = 2)
plot(roll_sd, main = "12-Month Rolling SD", ylab = "SD", col = "darkgreen")
```




```
par(mfrow = c(1, 1))
```

3.4 Formal Stationarity Tests

```r
# ADF: H0 = non-stationary (unit root present)
adf.test(starts_ts)
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  starts_ts
## Dickey-Fuller = -2.6716, Lag order = 4, p-value = 0.2975
## alternative hypothesis: stationary
```

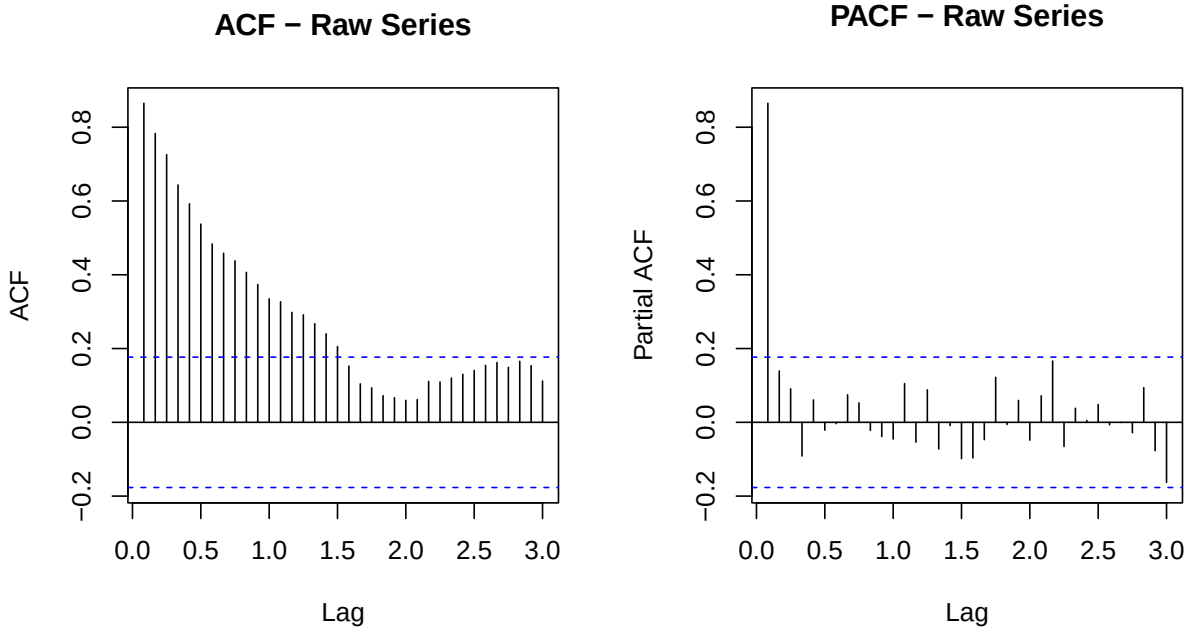
```r
# KPSS: H0 = stationary
kpss.test(starts_ts)
```

```
##
##  KPSS Test for Level Stationarity
##
## data:  starts_ts
## KPSS Level = 1.4458, Truncation lag parameter = 4, p-value = 0.01
```

---

4 Section 2 — ACF, PACF, EACF

```r
par(mfrow = c(1, 2))
acf(starts_ts,  lag.max = 36, main = "ACF - Raw Series")
pacf(starts_ts, lag.max = 36, main = "PACF - Raw Series")
```



```r
par(mfrow = c(1, 1))
```

```r
eacf(starts_ts)
```

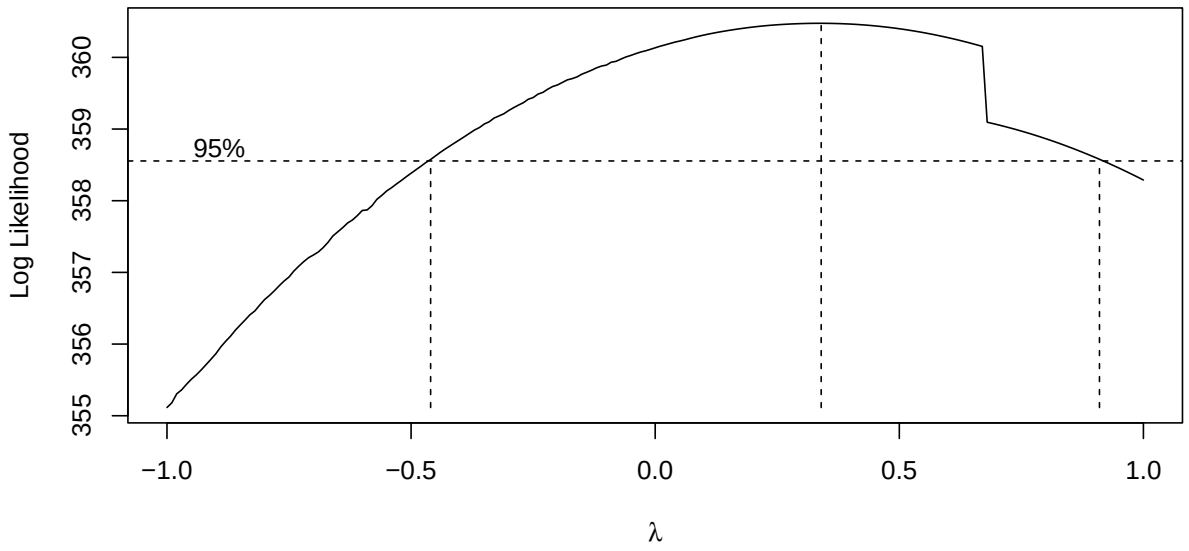
```
## AR/MA
##   0 1 2 3 4 5 6 7 8 9 10 11 12 13
## 0 x x x x x x x x x x x  x  x  x
## 1 x o o o o o o o o o o  o  o  o
## 2 x o o o o o o o o o o  o  o  o
## 3 x o o o o o o o o o o  o  o  o
## 4 x x o o o o o o o o o  o  o  o
## 5 o o o o o o o o o o o  o  o  o
## 6 x o o o o o o o o o o  o  o  o
## 7 o o x o o o o o o o o  o  o  o
```

---

5 Section 3 — Transformations

5.1 Box-Cox

```r
BC <- BoxCox.ar(starts_ts, lambda = seq(-1, 1, 0.01))
```




```r
lambda_opt <- BC$lambda[which.max(BC$loglike)]
cat("Optimal lambda:", round(lambda_opt, 3), "\n")
```

```
## Optimal lambda: 0.34
```

```
cat("If lambda near 1: no transformation needed.\n")
```

```
## If lambda near 1: no transformation needed.
```

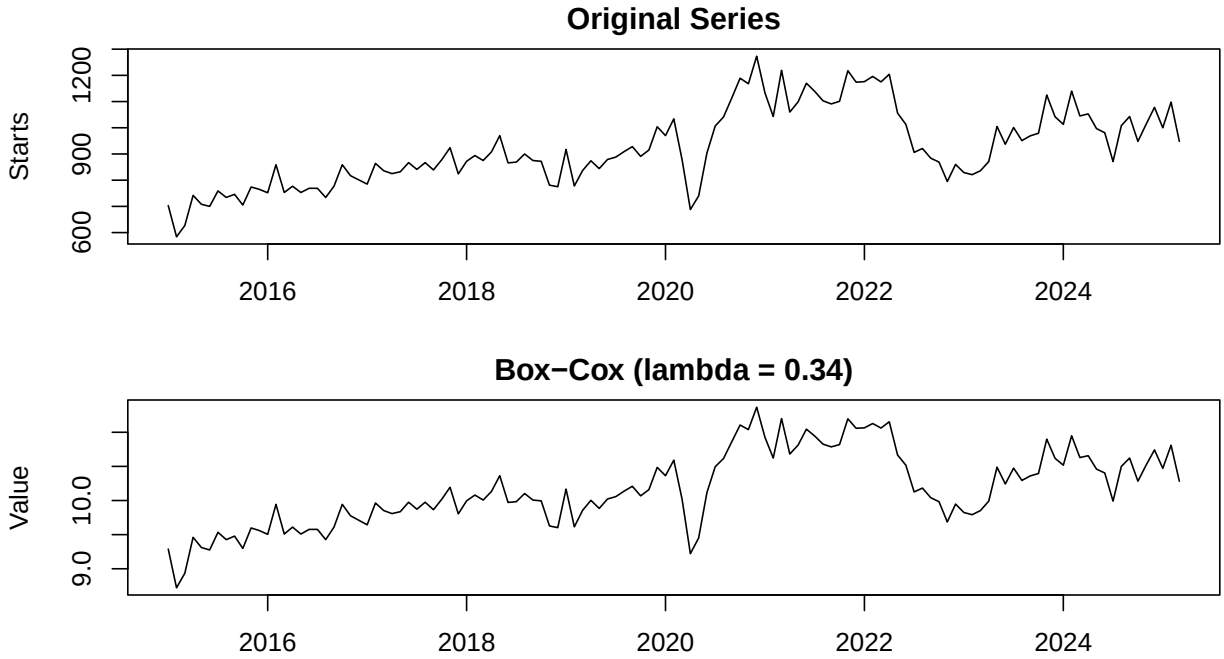
```
cat("If lambda near 0: log transformation appropriate.\n")
```

```
## If lambda near 0: log transformation appropriate.
```

```
if (abs(lambda_opt - 1) > 0.2 & abs(lambda_opt) < 0.05) {
  starts_tr <- log(starts_ts)
  tr_label  <- "Log-Transformed"
} else if (abs(lambda_opt - 1) > 0.2) {
  starts_tr <- starts_ts^lambda_opt
  tr_label  <- paste0("Box-Cox (lambda = ", round(lambda_opt, 2), ")")
} else {
  starts_tr <- starts_ts
  tr_label  <- "No transformation needed (lambda near 1)"
}
cat(tr_label, "\n")
```

```
## Box-Cox (lambda = 0.34)
```

```
par(mfrow = c(2, 1), mar = c(3, 4, 2, 1))
plot(starts_ts, main = "Original Series", ylab = "Starts")
plot(starts_tr, main = tr_label, ylab = "Value")
```

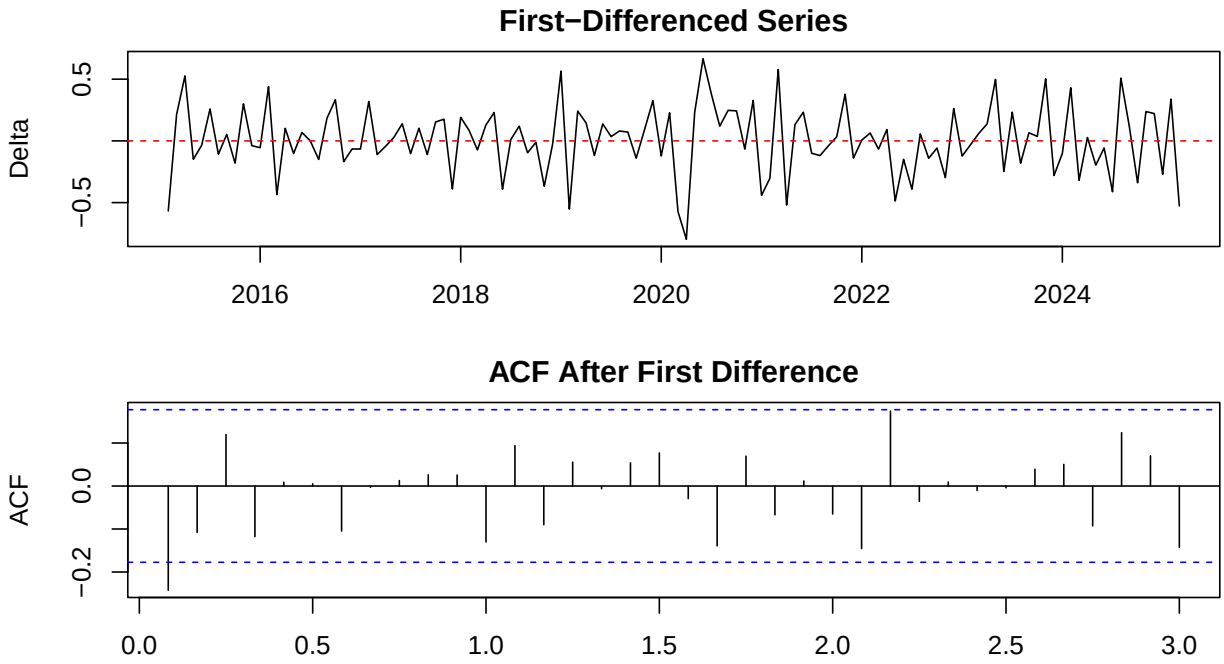


```r
par(mfrow = c(1, 1))
```

5.2 Differencing

```r
d1 <- diff(starts_tr, differences = 1)

par(mfrow = c(2, 1), mar = c(3, 4, 2, 1))
plot(d1, main = "First-Differenced Series", ylab = "Delta")
abline(h = 0, col = "red", lty = 2)
acf(d1, lag.max = 36, main = "ACF After First Difference")
```




```r
par(mfrow = c(1, 1))

adf.test(d1)
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  d1
## Dickey-Fuller = -5.5308, Lag order = 4, p-value = 0.01
## alternative hypothesis: stationary
```

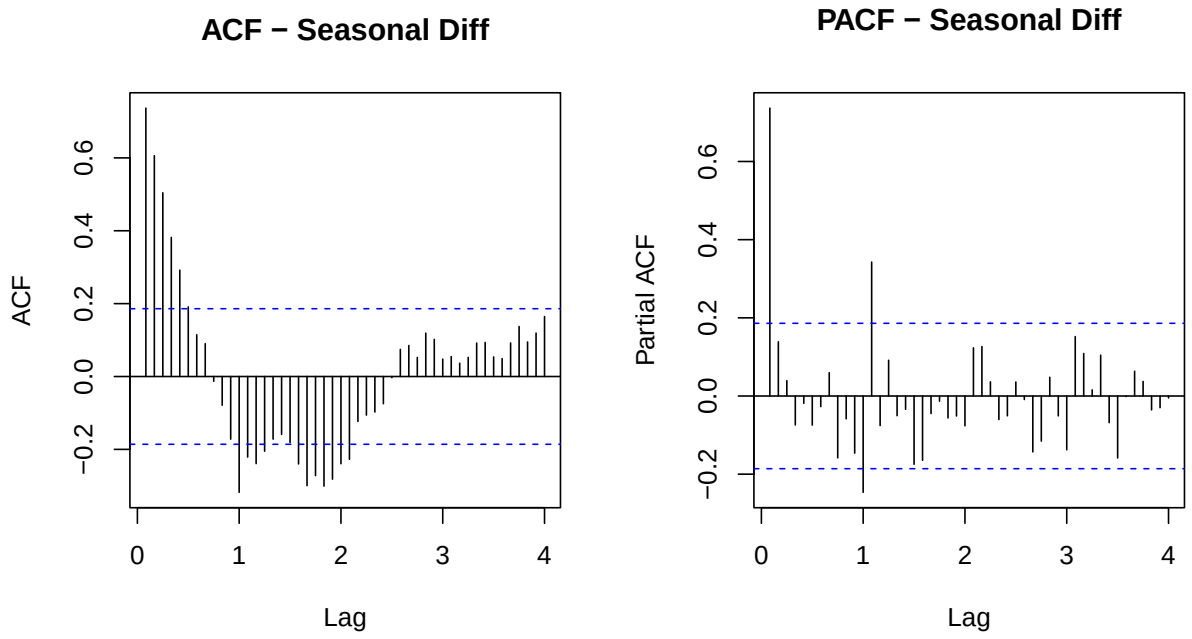
```r
kpss.test(d1)
```

```
##
##  KPSS Test for Level Stationarity
##
```

```
## data:  d1
## KPSS Level = 0.063763, Truncation lag parameter = 4, p-value = 0.1
```

```
# Note: FRED HOUST1F is already seasonally adjusted by Census Bureau.
# We attempt seasonal differencing to demonstrate the method, per course requirements.
d_seas <- diff(starts_tr, lag = 12)

par(mfrow = c(1, 2))
acf(d_seas,  lag.max = 48, main = "ACF - Seasonal Diff")
pacf(d_seas, lag.max = 48, main = "PACF - Seasonal Diff")
```



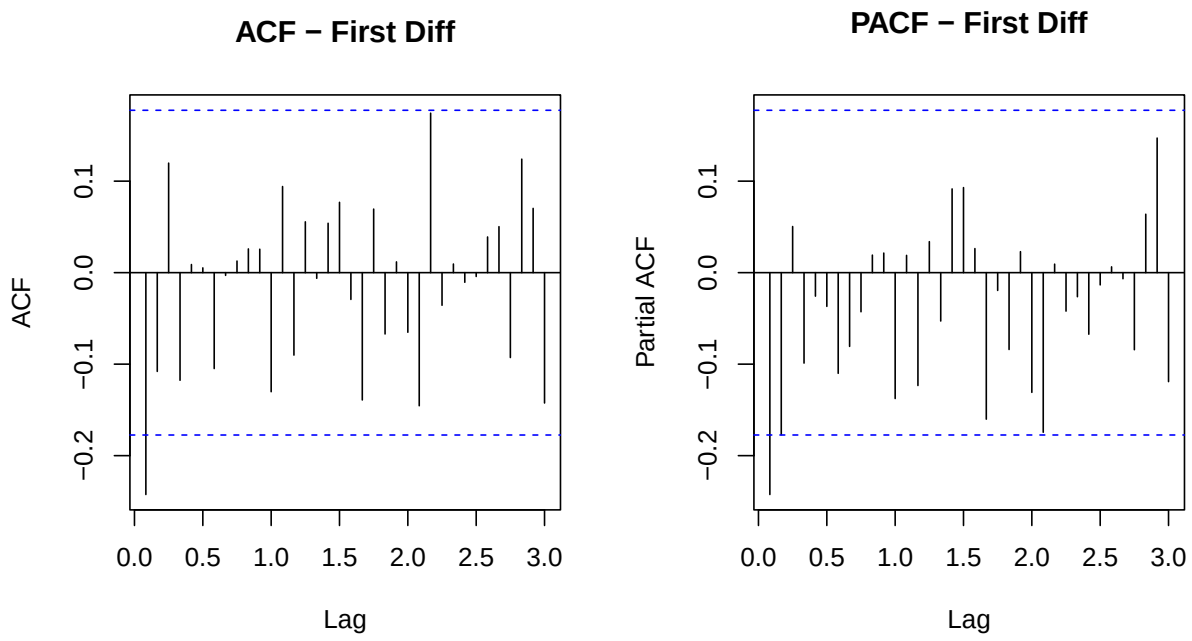

```
par(mfrow = c(1, 1))

adf.test(d_seas)
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  d_seas
## Dickey-Fuller = -2.9355, Lag order = 4, p-value = 0.1885
## alternative hypothesis: stationary
```

5.3 ACF / PACF / EACF on Differenced Series

```
par(mfrow = c(1, 2))
acf(d1,  lag.max = 36, main = "ACF - First Diff")
pacf(d1, lag.max = 36, main = "PACF - First Diff")
```




```
par(mfrow = c(1, 1))

eacf(d1)
```

```
## AR/MA
##   0 1 2 3 4 5 6 7 8 9 10 11 12 13
## 0 x o o o o o o o o o o  o  o  o
## 1 x x o o o o o o o o o  o  o  o
## 2 x o o o o o o o o o o  o  o  o
## 3 x x o o o o o o o o o  o  o  o
## 4 x x x o o o o o o o o  o  o  o
## 5 x o x x o o o o o o o  o  o  o
## 6 x o x o o o o o o o o  o  o  o
## 7 x o o o o o o o o o o  o  o  o
```

---

6 Section 4 — Model Identification

6.1 auto.arima

```
auto_mod <- auto.arima(starts_tr, seasonal = TRUE,
                       stepwise = FALSE, approximation = FALSE,
                       trace = TRUE)
```

```
##
##  ARIMA(0,1,0)                            : 31.65778
```

```
##  ARIMA(0,1,0)            with drift         : 33.61675
##  ARIMA(0,1,0)(0,0,1)[12]                    : 30.63967
##  ARIMA(0,1,0)(0,0,1)[12] with drift         : 32.55775
##  ARIMA(0,1,0)(0,0,2)[12]                    : 30.00145
##  ARIMA(0,1,0)(0,0,2)[12] with drift         : 31.80723
##  ARIMA(0,1,0)(1,0,0)[12]                    : 31.37814
##  ARIMA(0,1,0)(1,0,0)[12] with drift         : 33.32544
##  ARIMA(0,1,0)(1,0,1)[12]                    : Inf
##  ARIMA(0,1,0)(1,0,1)[12] with drift         : Inf
##  ARIMA(0,1,0)(1,0,2)[12]                    : Inf
##  ARIMA(0,1,0)(1,0,2)[12] with drift         : Inf
##  ARIMA(0,1,0)(2,0,0)[12]                    : 32.38088
##  ARIMA(0,1,0)(2,0,0)[12] with drift         : 34.34519
##  ARIMA(0,1,0)(2,0,1)[12]                    : Inf
##  ARIMA(0,1,0)(2,0,1)[12] with drift         : Inf
##  ARIMA(0,1,0)(2,0,2)[12]                    : Inf
##  ARIMA(0,1,0)(2,0,2)[12] with drift         : Inf
##  ARIMA(0,1,1)                               : 23.6031
##  ARIMA(0,1,1)            with drift         : 25.2733
##  ARIMA(0,1,1)(0,0,1)[12]                    : 23.05646
##  ARIMA(0,1,1)(0,0,1)[12] with drift         : 24.55499
##  ARIMA(0,1,1)(0,0,2)[12]                    : 21.82361
##  ARIMA(0,1,1)(0,0,2)[12] with drift         : 22.9879
##  ARIMA(0,1,1)(1,0,0)[12]                    : 23.8867
##  ARIMA(0,1,1)(1,0,0)[12] with drift         : 25.48018
##  ARIMA(0,1,1)(1,0,1)[12]                    : Inf
##  ARIMA(0,1,1)(1,0,1)[12] with drift         : Inf
##  ARIMA(0,1,1)(1,0,2)[12]                    : 22.08219
##  ARIMA(0,1,1)(1,0,2)[12] with drift         : Inf
##  ARIMA(0,1,1)(2,0,0)[12]                    : 23.69088
##  ARIMA(0,1,1)(2,0,0)[12] with drift         : 25.20428
##  ARIMA(0,1,1)(2,0,1)[12]                    : 22.07502
##  ARIMA(0,1,1)(2,0,1)[12] with drift         : 23.00788
##  ARIMA(0,1,1)(2,0,2)[12]                    : 24.28806
##  ARIMA(0,1,1)(2,0,2)[12] with drift         : 25.2593
##  ARIMA(0,1,2)                               : 25.04111
##  ARIMA(0,1,2)            with drift         : 26.65745
##  ARIMA(0,1,2)(0,0,1)[12]                    : 24.2851
##  ARIMA(0,1,2)(0,0,1)[12] with drift         : 25.63394
##  ARIMA(0,1,2)(0,0,2)[12]                    : 23.44979
##  ARIMA(0,1,2)(0,0,2)[12] with drift         : 24.42033
##  ARIMA(0,1,2)(1,0,0)[12]                    : 25.14366
##  ARIMA(0,1,2)(1,0,0)[12] with drift         : 26.62654
##  ARIMA(0,1,2)(1,0,1)[12]                    : Inf
##  ARIMA(0,1,2)(1,0,1)[12] with drift         : Inf
##  ARIMA(0,1,2)(1,0,2)[12]                    : 23.89792
##  ARIMA(0,1,2)(1,0,2)[12] with drift         : 24.61576
##  ARIMA(0,1,2)(2,0,0)[12]                    : 25.2525
##  ARIMA(0,1,2)(2,0,0)[12] with drift         : 26.64793
##  ARIMA(0,1,2)(2,0,1)[12]                    : 23.88926
##  ARIMA(0,1,2)(2,0,1)[12] with drift         : 24.60624
##  ARIMA(0,1,3)                               : 26.64265
##  ARIMA(0,1,3)            with drift         : 28.3957
##  ARIMA(0,1,3)(0,0,1)[12]                    : 25.47712
```

```
##  ARIMA(0,1,3)(0,0,1)[12] with drift         : 27.07098
##  ARIMA(0,1,3)(0,0,2)[12]                    : 24.93634
##  ARIMA(0,1,3)(0,0,2)[12] with drift         : 26.23864
##  ARIMA(0,1,3)(1,0,0)[12]                    : 26.43379
##  ARIMA(0,1,3)(1,0,0)[12] with drift         : 28.11983
##  ARIMA(0,1,3)(1,0,1)[12]                    : Inf
##  ARIMA(0,1,3)(1,0,1)[12] with drift         : Inf
##  ARIMA(0,1,3)(2,0,0)[12]                    : 26.85166
##  ARIMA(0,1,3)(2,0,0)[12] with drift         : 28.45934
##  ARIMA(0,1,4)                               : 26.55343
##  ARIMA(0,1,4)            with drift         : 28.06879
##  ARIMA(0,1,4)(0,0,1)[12]                    : 24.50537
##  ARIMA(0,1,4)(0,0,1)[12] with drift         : 25.44548
##  ARIMA(0,1,4)(1,0,0)[12]                    : 25.88639
##  ARIMA(0,1,4)(1,0,0)[12] with drift         : 27.15082
##  ARIMA(0,1,5)                               : 28.69545
##  ARIMA(0,1,5)            with drift         : 30.18931
##  ARIMA(1,1,0)                               : 26.00294
##  ARIMA(1,1,0)            with drift         : 27.8303
##  ARIMA(1,1,0)(0,0,1)[12]                    : 25.53351
##  ARIMA(1,1,0)(0,0,1)[12] with drift         : 27.26294
##  ARIMA(1,1,0)(0,0,2)[12]                    : 24.16811
##  ARIMA(1,1,0)(0,0,2)[12] with drift         : 25.6574
##  ARIMA(1,1,0)(1,0,0)[12]                    : 26.32642
##  ARIMA(1,1,0)(1,0,0)[12] with drift         : 28.1157
##  ARIMA(1,1,0)(1,0,1)[12]                    : Inf
##  ARIMA(1,1,0)(1,0,1)[12] with drift         : Inf
##  ARIMA(1,1,0)(1,0,2)[12]                    : Inf
##  ARIMA(1,1,0)(1,0,2)[12] with drift         : Inf
##  ARIMA(1,1,0)(2,0,0)[12]                    : 26.29032
##  ARIMA(1,1,0)(2,0,0)[12] with drift         : 28.05053
##  ARIMA(1,1,0)(2,0,1)[12]                    : Inf
##  ARIMA(1,1,0)(2,0,1)[12] with drift         : Inf
##  ARIMA(1,1,0)(2,0,2)[12]                    : Inf
##  ARIMA(1,1,0)(2,0,2)[12] with drift         : Inf
##  ARIMA(1,1,1)                               : 25.18347
##  ARIMA(1,1,1)            with drift         : 26.72847
##  ARIMA(1,1,1)(0,0,1)[12]                    : 24.51536
##  ARIMA(1,1,1)(0,0,1)[12] with drift         : 25.5535
##  ARIMA(1,1,1)(0,0,2)[12]                    : 23.57449
##  ARIMA(1,1,1)(0,0,2)[12] with drift         : 23.96121
##  ARIMA(1,1,1)(1,0,0)[12]                    : 25.36773
##  ARIMA(1,1,1)(1,0,0)[12] with drift         : 26.61929
##  ARIMA(1,1,1)(1,0,1)[12]                    : Inf
##  ARIMA(1,1,1)(1,0,1)[12] with drift         : Inf
##  ARIMA(1,1,1)(1,0,2)[12]                    : 24.02202
##  ARIMA(1,1,1)(1,0,2)[12] with drift         : Inf
##  ARIMA(1,1,1)(2,0,0)[12]                    : 25.36426
##  ARIMA(1,1,1)(2,0,0)[12] with drift         : 26.50634
##  ARIMA(1,1,1)(2,0,1)[12]                    : 24.01322
##  ARIMA(1,1,1)(2,0,1)[12] with drift         : Inf
##  ARIMA(1,1,2)                               : Inf
##  ARIMA(1,1,2)            with drift         : Inf
##  ARIMA(1,1,2)(0,0,1)[12]                    : Inf
```

```
##  ARIMA(1,1,2)(0,0,1)[12] with drift         : Inf
##  ARIMA(1,1,2)(0,0,2)[12]                    : Inf
##  ARIMA(1,1,2)(0,0,2)[12] with drift         : Inf
##  ARIMA(1,1,2)(1,0,0)[12]                    : Inf
##  ARIMA(1,1,2)(1,0,0)[12] with drift         : Inf
##  ARIMA(1,1,2)(1,0,1)[12]                    : Inf
##  ARIMA(1,1,2)(1,0,1)[12] with drift         : Inf
##  ARIMA(1,1,2)(2,0,0)[12]                    : Inf
##  ARIMA(1,1,2)(2,0,0)[12] with drift         : Inf
##  ARIMA(1,1,3)                               : Inf
##  ARIMA(1,1,3)            with drift         : Inf
##  ARIMA(1,1,3)(0,0,1)[12]                    : Inf
##  ARIMA(1,1,3)(0,0,1)[12] with drift         : Inf
##  ARIMA(1,1,3)(1,0,0)[12]                    : Inf
##  ARIMA(1,1,3)(1,0,0)[12] with drift         : Inf
##  ARIMA(1,1,4)                               : 27.58531
##  ARIMA(1,1,4)            with drift         : Inf
##  ARIMA(2,1,0)                               : 24.2903
##  ARIMA(2,1,0)            with drift         : 25.99008
##  ARIMA(2,1,0)(0,0,1)[12]                    : 23.26058
##  ARIMA(2,1,0)(0,0,1)[12] with drift         : 24.7471
##  ARIMA(2,1,0)(0,0,2)[12]                    : 22.43522
##  ARIMA(2,1,0)(0,0,2)[12] with drift         : 23.61702
##  ARIMA(2,1,0)(1,0,0)[12]                    : 24.19587
##  ARIMA(2,1,0)(1,0,0)[12] with drift         : 25.79191
##  ARIMA(2,1,0)(1,0,1)[12]                    : Inf
##  ARIMA(2,1,0)(1,0,1)[12] with drift         : Inf
##  ARIMA(2,1,0)(1,0,2)[12]                    : 22.95588
##  ARIMA(2,1,0)(1,0,2)[12] with drift         : 23.89793
##  ARIMA(2,1,0)(2,0,0)[12]                    : 24.3198
##  ARIMA(2,1,0)(2,0,0)[12] with drift         : 25.85857
##  ARIMA(2,1,0)(2,0,1)[12]                    : 22.94957
##  ARIMA(2,1,0)(2,0,1)[12] with drift         : 23.89123
##  ARIMA(2,1,1)                               : 25.40025
##  ARIMA(2,1,1)            with drift         : 27.19344
##  ARIMA(2,1,1)(0,0,1)[12]                    : 23.50294
##  ARIMA(2,1,1)(0,0,1)[12] with drift         : 25.08162
##  ARIMA(2,1,1)(0,0,2)[12]                    : 22.7742
##  ARIMA(2,1,1)(0,0,2)[12] with drift         : 24.08224
##  ARIMA(2,1,1)(1,0,0)[12]                    : 24.74793
##  ARIMA(2,1,1)(1,0,0)[12] with drift         : 26.44642
##  ARIMA(2,1,1)(1,0,1)[12]                    : 21.21611
##  ARIMA(2,1,1)(1,0,1)[12] with drift         : Inf
##  ARIMA(2,1,1)(2,0,0)[12]                    : 24.91495
##  ARIMA(2,1,1)(2,0,0)[12] with drift         : Inf
##  ARIMA(2,1,2)                               : 27.57375
##  ARIMA(2,1,2)            with drift         : 29.39379
##  ARIMA(2,1,2)(0,0,1)[12]                    : 25.69685
##  ARIMA(2,1,2)(0,0,1)[12] with drift         : 27.33214
##  ARIMA(2,1,2)(1,0,0)[12]                    : Inf
##  ARIMA(2,1,2)(1,0,0)[12] with drift         : Inf
##  ARIMA(2,1,3)                               : 27.34034
##  ARIMA(2,1,3)            with drift         : 29.14481
##  ARIMA(3,1,0)                               : 25.93028
```

```
##  ARIMA(3,1,0)            with drift         : 27.72501
##  ARIMA(3,1,0)(0,0,1)[12]                    : 24.45662
##  ARIMA(3,1,0)(0,0,1)[12] with drift         : 26.08352
##  ARIMA(3,1,0)(0,0,2)[12]                    : 23.54964
##  ARIMA(3,1,0)(0,0,2)[12] with drift         : 24.9314
##  ARIMA(3,1,0)(1,0,0)[12]                    : 25.56598
##  ARIMA(3,1,0)(1,0,0)[12] with drift         : 27.2873
##  ARIMA(3,1,0)(1,0,1)[12]                    : Inf
##  ARIMA(3,1,0)(1,0,1)[12] with drift         : Inf
##  ARIMA(3,1,0)(2,0,0)[12]                    : 25.671
##  ARIMA(3,1,0)(2,0,0)[12] with drift         : 27.3524
##  ARIMA(3,1,1)                               : 27.57497
##  ARIMA(3,1,1)            with drift         : 29.40262
##  ARIMA(3,1,1)(0,0,1)[12]                    : 25.70952
##  ARIMA(3,1,1)(0,0,1)[12] with drift         : 27.33318
##  ARIMA(3,1,1)(1,0,0)[12]                    : 26.9603
##  ARIMA(3,1,1)(1,0,0)[12] with drift         : 28.69793
##  ARIMA(3,1,2)                               : 27.54514
##  ARIMA(3,1,2)            with drift         : Inf
##  ARIMA(4,1,0)                               : 26.98348
##  ARIMA(4,1,0)            with drift         : 28.73591
##  ARIMA(4,1,0)(0,0,1)[12]                    : 24.91011
##  ARIMA(4,1,0)(0,0,1)[12] with drift         : 26.36759
##  ARIMA(4,1,0)(1,0,0)[12]                    : 26.27815
##  ARIMA(4,1,0)(1,0,0)[12] with drift         : 27.89954
##  ARIMA(4,1,1)                               : 27.34778
##  ARIMA(4,1,1)            with drift         : Inf
##  ARIMA(5,1,0)                               : 29.15529
##  ARIMA(5,1,0)            with drift         : 30.9293
##
##
##
##  Best model: ARIMA(2,1,1)(1,0,1)[12]
```

```
summary(auto_mod)
```

```
## Series: starts_tr
## ARIMA(2,1,1)(1,0,1)[12]
##
## Coefficients:
##          ar1      ar2     ma1    sar1     sma1
##       -0.8052  -0.3063  0.5490  0.6164  -0.8797
## s.e.   0.2757   0.0957  0.2822  0.2275   0.2408
##
## sigma^2 = 0.06298:  log likelihood = -4.24
## AIC=20.49   AICc=21.22   BIC=37.31
##
## Training set error measures:
##                      ME      RMSE       MAE       MPE     MAPE      MASE
## Training set 0.02609433 0.2447568 0.1843313 0.2211279 1.811058 0.4218696
##                      ACF1
## Training set -0.002286239
```

6.2 Candidate Models

```
m1 <- arima(starts_tr, order = c(1, 1, 1), method = "CSS-ML")
m2 <- arima(starts_tr, order = c(2, 1, 1), method = "CSS-ML")
m3 <- arima(starts_tr, order = c(1, 1, 2), method = "CSS-ML")

cat("ARIMA(1,1,1) AIC:", round(AIC(m1), 2), "| BIC:", round(BIC(m1), 2), "\n")
```

```
## ARIMA(1,1,1) AIC: 24.98 | BIC: 33.39
```

```
cat("ARIMA(2,1,1) AIC:", round(AIC(m2), 2), "| BIC:", round(BIC(m2), 2), "\n")
```

```
## ARIMA(2,1,1) AIC: 25.06 | BIC: 36.27
```

```
cat("ARIMA(1,1,2) AIC:", round(AIC(m3), 2), "| BIC:", round(BIC(m3), 2), "\n")
```

```
## ARIMA(1,1,2) AIC: 21.95 | BIC: 33.17
```

```
cat("auto.arima  AIC:", round(AIC(auto_mod), 2), "| BIC:", round(BIC(auto_mod), 2), "\n")
```

```
## auto.arima  AIC: 20.49 | BIC: 37.31
```

```
coeftest(m1)
```

```
##
## z test of coefficients:
##
##     Estimate Std. Error z value Pr(>|z|)
## ar1  0.26639    0.44501  0.5986   0.5494
## ma1 -0.56069    0.39513 -1.4190   0.1559
```

```
coeftest(auto_mod)
```

```
##
## z test of coefficients:
##
##       Estimate Std. Error z value  Pr(>|z|)
## ar1  -0.805157   0.275741 -2.9200 0.0035006 **
## ar2  -0.306281   0.095712 -3.2000 0.0013742 **
## ma1   0.549030   0.282219  1.9454 0.0517268 .
## sar1  0.616414   0.227510  2.7094 0.0067406 **
## sma1 -0.879720   0.240850 -3.6526 0.0002596 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

6.3 CSS vs ML Comparison

```r
m_css <- arima(starts_tr, order = c(1, 1, 1), method = "CSS")
m_ml  <- arima(starts_tr, order = c(1, 1, 1), method = "ML")

cat("CSS:\n"); print(round(coef(m_css), 4))
```

```
## CSS:

##     ar1     ma1
## -0.0743 -0.2195
```

```r
cat("ML:\n");  print(round(coef(m_ml),  4))
```

```
## ML:

##     ar1     ma1
##  0.2665 -0.5608
```

```r
# Use auto.arima result as final model
final_model <- auto_mod
```

---

7 Section 5 — Stationarity and Invertibility

```r
ar_phi <- final_model$model$phi
if (length(ar_phi) > 0) {
  ar_roots <- polyroot(c(1, -ar_phi))
  cat("AR root moduli:", round(Mod(ar_roots), 4), "\n")
  cat("Stationary (all > 1)?", all(Mod(ar_roots) > 1), "\n\n")
} else {
  cat("No AR component.\n\n")
}
```

```
## AR root moduli: 1.0411 1.0411 1.0411 1.0411 1.0411 1.0411 1.0411 1.0411 1.0411 1.0411 1.0411 1.0411 1
## Stationary (all > 1)? TRUE
```

```r
ma_theta <- final_model$model$theta
if (length(ma_theta) > 0) {
  ma_roots <- polyroot(c(1, ma_theta))
  cat("MA root moduli:", round(Mod(ma_roots), 4), "\n")
  cat("Invertible (all > 1)?", all(Mod(ma_roots) > 1), "\n")
} else {
  cat("No MA component.\n")
}
```

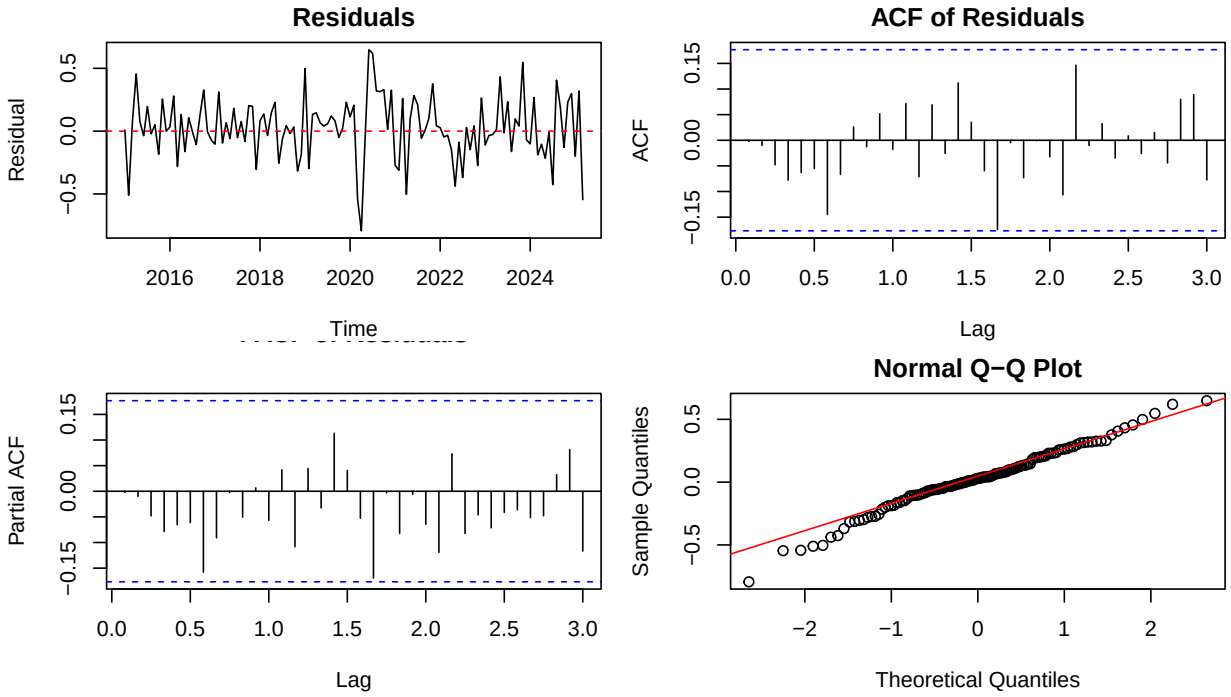
```
## MA root moduli: 1.0107 1.0107 1.0107 1.0107 1.0107 1.0107 1.0107 1.0107 1.0107 1.0107 1.0107 1.0107 1
## Invertible (all > 1)? TRUE
```

---

8 Section 6 — Residual Diagnostics

```
res <- residuals(final_model)

par(mfrow = c(2, 2), mar = c(4, 4, 2, 1))
plot(res, main = "Residuals", ylab = "Residual", type = "l")
abline(h = 0, col = "red", lty = 2)
acf(res,  lag.max = 36, main = "ACF of Residuals")
pacf(res, lag.max = 36, main = "PACF of Residuals")
qqnorm(res); qqline(res, col = "red")
```




```
par(mfrow = c(1, 1))

# Ljung-Box: H0 = residuals are white noise
Box.test(res, lag = 12, type = "Ljung-Box", fitdf = length(final_model$coef))
```

```
##
##  Box-Ljung test
##
## data:  res
## X-squared = 5.8966, df = 7, p-value = 0.5519
```

```
# Shapiro-Wilk normality
shapiro.test(as.numeric(res))
```

```
##
##  Shapiro-Wilk normality test
##
```

```
## data:  as.numeric(res)
## W = 0.98454, p-value = 0.174
```

---

9 Section 7 — Train / Test Split

```r
# Hold out last 12 months
# Use window() to preserve ts class (head/tail strips it)
n        <- length(starts_tr)
train_ts <- window(starts_tr, end   = time(starts_tr)[n - 12])
test_ts  <- window(starts_tr, start = time(starts_tr)[n - 11])

cat("Train:", length(train_ts), "obs\n")
```

```
## Train: 111 obs
```

```r
cat("Test: ", length(test_ts),  "obs\n")
```

```
## Test:  12 obs
```

```r
# Pull order from final_model safely
p <- final_model$arma[1]
q <- final_model$arma[2]
P <- final_model$arma[3]
Q <- final_model$arma[4]
d <- final_model$arma[6]
D <- final_model$arma[7]
s <- final_model$arma[5]

cat("Model order - p:", p, "d:", d, "q:", q,
    "| P:", P, "D:", D, "Q:", Q, "s:", s, "\n")
```

```
## Model order - p: 2 d: 1 q: 1 | P: 1 D: 0 Q: 1 s: 12
```

```r
# Use Arima() from forecast package - more robust than arima() for refitting
if (s > 1) {
  train_model <- Arima(train_ts,
                       order    = c(p, d, q),
                       seasonal = list(order = c(P, D, Q), period = s))
} else {
  train_model <- Arima(train_ts, order = c(p, d, q))
}
print(train_model)
```

```
## Series: train_ts
## ARIMA(2,1,1)(1,0,1)[12]
##
```

```
## Coefficients:
##           ar1      ar2    ma1    sar1     sma1
##       -1.1066  -0.2557  1.000  0.6209  -0.9999
## s.e.   0.0947   0.0952  0.044  0.1019   0.2311
##
## sigma^2 = 0.05177:  log likelihood = 0.54
## AIC=10.93   AICc=11.74   BIC=27.13
```

```
fc_test <- forecast(train_model, h = 12, level = 95)

# Back-transform
if (tr_label == "Log-Transformed") {
  fc_vals  <- exp(as.numeric(fc_test$mean))
  act_vals <- exp(as.numeric(test_ts))
} else if (tr_label != "No transformation needed (lambda near 1)") {
  fc_vals  <- as.numeric(fc_test$mean)^(1 / lambda_opt)
  act_vals <- as.numeric(test_ts)^(1 / lambda_opt)
} else {
  fc_vals  <- as.numeric(fc_test$mean)
  act_vals <- as.numeric(test_ts)
}

mae  <- mean(abs(fc_vals - act_vals))
rmse <- sqrt(mean((fc_vals - act_vals)^2))
mape <- mean(abs((fc_vals - act_vals) / act_vals)) * 100

cat("MAE: ",  round(mae,  2), "\n")
```

```
## MAE:  67.97
```

```
cat("RMSE:",  round(rmse, 2), "\n")
```

```
## RMSE: 83.07
```

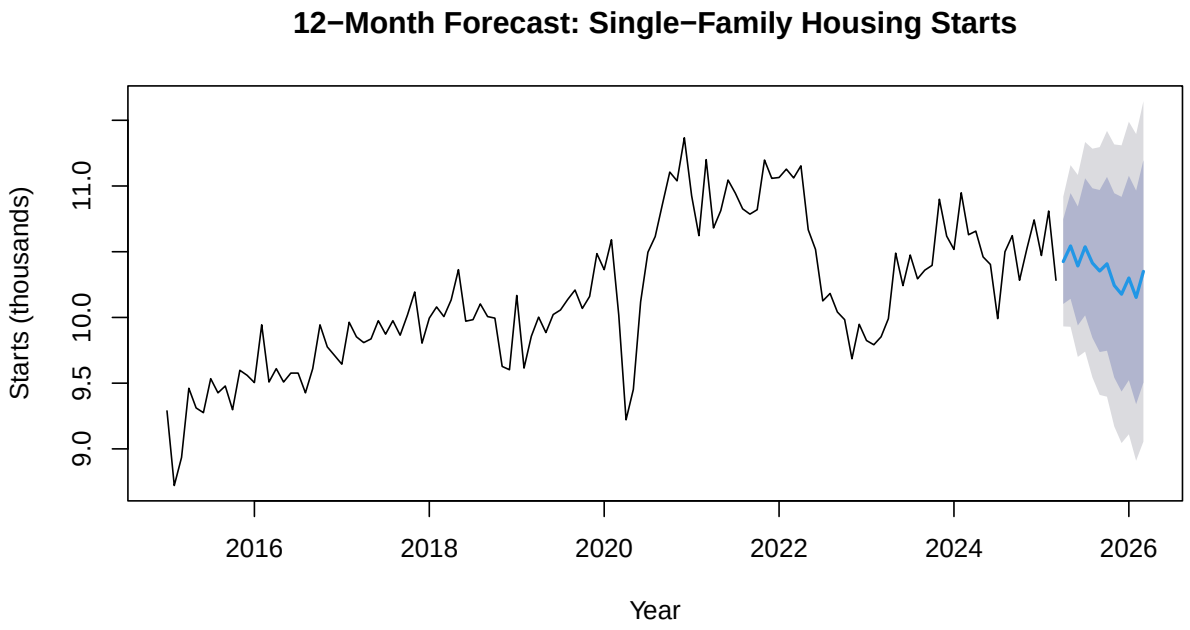
```
cat("MAPE:",  round(mape, 2), "%\n")
```

```
## MAPE: 6.91 %
```

---

10 Section 8 — 12-Month Forward Forecast

```
fc_final <- forecast(final_model, h = 12, level = c(80, 95))
plot(fc_final,
     main = "12-Month Forecast: Single-Family Housing Starts",
     ylab = "Starts (thousands)", xlab = "Year")
```




```
fc_dates <- seq(max(housing$date) %m+% months(1),
                by = "month", length.out = 12)

back_tr <- function(x) {
  if (tr_label == "Log-Transformed") return(exp(x))
  if (tr_label != "No transformation needed (lambda near 1)") return(x^(1/lambda_opt))
  return(x)
}

fc_df <- data.frame(
  Month    = format(fc_dates, "%b %Y"),
  Forecast = round(back_tr(as.numeric(fc_final$mean)), 1),
  Lo_95    = round(back_tr(as.numeric(fc_final$lower[, 2])), 1),
  Hi_95    = round(back_tr(as.numeric(fc_final$upper[, 2])), 1)
)
kable(fc_df, col.names = c("Month", "Forecast", "Lower 95%", "Upper 95%"))
```

Month	Forecast	Lower 95%	Upper 95%
Apr 2025	987.1	855.9	1130.9
May 2025	1020.5	855.2	1205.7
Jun 2025	977.8	798.4	1182.0
Jul 2025	1018.7	808.2	1262.4
Aug 2025	984.0	761.3	1245.9
Sep 2025	967.2	730.4	1249.7
Oct 2025	982.3	727.3	1290.2
Nov 2025	937.4	677.1	1256.4
Dec 2025	919.5	649.9	1254.2
Jan 2026	952.7	664.2	1313.6
Feb 2026	913.0	621.9	1282.3

Month	Forecast	Lower 95%	Upper 95%
Mar 2026	966.4	652.4	1366.6

---

11 Section 9 — Pre vs Post Rate Hike Comparison

```
pre  <- window(starts_ts, end   = c(2022, 2))
post <- window(starts_ts, start = c(2023, 8))

m_pre  <- arima(pre,  order = c(1, 0, 0), method = "CSS-ML")
m_post <- arima(post, order = c(1, 0, 0), method = "CSS-ML")

cat("AR(1) phi before rate hikes:", round(coef(m_pre)["ar1"],  4), "\n")
```

```
## AR(1) phi before rate hikes: 0.9092
```

```
cat("AR(1) phi after rate hikes: ", round(coef(m_post)["ar1"], 4), "\n")
```

```
## AR(1) phi after rate hikes:  0.0551
```